
System Design Notes

High Level Design (HLD) + Low Level Design (LLD)

Comprehensive study guide covering architecture, databases, networking, caching, microservices, observability, security, and code-level design. Running example: WhatsApp-like chat system.

Table of Contents

Introduction

- What is System Design?
- HLD vs LLD
- System Design Approach — 6 Steps
- Functional vs Non-Functional Requirements
- Back-of-the-Envelope Estimations

High Level Design (HLD)

- How Does the Internet Work?
- Threads
- Pages & Virtual Memory
- Horizontal vs Vertical Scaling
- Serverful vs Serverless
- SQL vs NoSQL Databases
- In-Memory Databases
- Data Replication & Migration
- Data Partitioning & Sharding
- Data Consistency & Isolation Levels
- CAP Theorem
- Consistency vs Availability
- Caching — Redis, Memcached & Policies
- Content Delivery Networks (CDN)
- TCP & UDP — Transport Layer
- HTTP/1, HTTP/2, HTTP/3 & HTTPS
- WebSockets & Real-Time Alternatives
- WebRTC & Video Streaming
- Load Balancing
- Consistent Hashing
- Proxy vs Reverse Proxy
- Rate Limiting
- Message Queues & Async Processing
- Event-Driven Architecture & Saga Pattern
- REST vs gRPC vs GraphQL
- Idempotency

- Monolith vs Microservices
- Migration to Microservices
- Single Point of Failure (SPOF)
- Cascading Failures, Circuit Breaker & Bulkhead
- Docker & Containerization
- Object Storage (S3)
- Logging
- Metrics, Monitoring & Distributed Tracing
- Anomaly Detection & Disaster Recovery
- Security — Auth, Encryption & Access Control
- Push vs Pull
- System Design Trade-offs
- Popular HLD Systems to Practice
- Search Autocomplete — HLD Glimpse

Low Level Design (LLD)

- OOP — Object-Oriented Programming
- SOLID Principles
- Coupling, Cohesion & Composition
- Design Patterns
- Concurrency & Thread Safety
- UML Diagrams
- API Design — Request/Response Modeling
- Clean Code & Avoiding God Classes
- MVC, MVP, MVVM & Repository Pattern
- Schema Design, Testability & Domain Modeling
- Common LLD Problems to Practice
- HLD vs LLD — Same Problem, Two Views

Study Guide & Summary

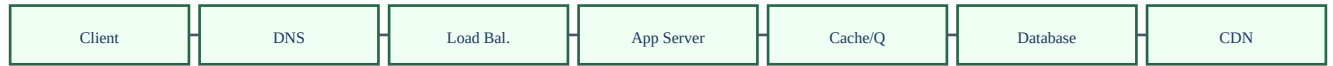
- Study Roadmap & Resources
- Summary & Key Takeaways

Visual Flow Diagram

Core flows on one page — read each row left to right. Top three: interview, request path, migration. Below: data, performance, networking, and async/LLD connectivity.



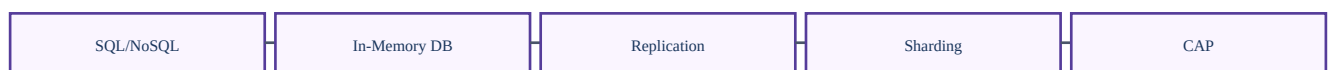
System Design Interview Flow



Chat Message / API Request Path



Monolith to Microservices (Strangler Fig)



Data Layer — Storage, Replication & Consistency



Performance & Traffic Routing



Networking, Real-Time & Delivery



Async, APIs, Reliability & LLD

Introduction

What is System Design?

System Design is the process of defining architecture, components, modules, interfaces, and data for a system to meet specified requirements. It answers one core question: how do we build software that works at real-world scale — millions of users, high availability, and maintainable code?

It spans two levels. HLD (High Level Design) covers how major components connect — services, databases, queues, load balancers. LLD (Low Level Design) covers how code is structured internally — classes, design patterns, APIs, and concurrency.

Throughout this guide we use a running example: a WhatsApp-like chat system.

- Used in product engineering, architecture reviews, and technical interviews.
- Good design balances scalability, reliability, cost, and simplicity.

HLD vs LLD

HLD and LLD are two complementary views of the same system. HLD focuses on architecture and infrastructure — what to build and how parts connect. LLD focuses on code-level structure — how each part is implemented.

Both are required in real projects: architects produce HLD diagrams; engineers produce LLD class diagrams and APIs.

Aspect	HLD	LLD
Focus	Architecture & component interaction	Classes, modules & patterns
Question	What are the parts? How do they connect?	How is each part built internally?
Artifacts	Architecture diagrams, DB & infra choices	Class/sequence diagrams, code structure
Example (Chat)	Chat svc + Kafka + Redis + Cassandra	Message, Chat, User + Repository pattern

System Design Approach — 6 Steps

Never jump straight to databases or microservices. Follow a structured process for every design problem — whether in interviews or real architecture reviews.

- 1. Requirements** — Gather functional requirements (features) and non-functional requirements (scale, latency, availability, security). For chat: send messages, 1B users, <200ms delivery.
- 2. Estimations** — Back-of-envelope math: DAU, QPS, storage, bandwidth. Estimations guide whether you need sharding, caching, or CDN.
- 3. API Design** — Define endpoints and contracts: POST /messages, WebSocket /connect, request/response shapes.
- 4. High-Level Design** — Draw components and data flow: Client → Load Balancer → Chat Service → Queue → Database.
- 5. Deep Dive** — Detail bottlenecks: shard by chat_id, cache recent messages in Redis, handle WebSocket scaling.
- 6. Trade-offs** — Explicitly justify choices: AP over CP, push vs pull, SQL vs NoSQL, monolith vs microservices.

Functional vs Non-Functional Requirements

Every system has two types of requirements. Functional requirements describe what the system must do — features and behaviors. Non-functional requirements describe how well it must perform — scale, speed, reliability, and security.

Both must be captured before designing architecture. Missing non-functional requirements leads to systems that work in demos but fail in production.

Functional Requirements

- Send and receive text messages
- Create and manage group chats
- Share media (photos, videos, documents)
- Read receipts and online status
- End-to-end encryption for privacy

Non-Functional Requirements

- Scale: support 1B+ users, 50B messages/day
- Latency: message delivery under 200ms
- Availability: 99.99% uptime — always on
- Reliability: zero message loss on crash
- Security: auth tokens, encrypted transit and at rest

Back-of-the-Envelope Estimations

Estimations prevent over-engineering and under-provisioning. Before choosing infrastructure, do quick math on users, QPS (Queries Per Second), storage, and bandwidth.

QPS measures how many read/write requests the system handles each second. Throughput is total data processed over time (e.g. MB/sec). CDN (Content Delivery Network) stores media copies on edge servers near users.

- DAU: 20% of 1B users = 200M daily active users
- Messages: $200M \times 50 \text{ msgs/day} = 10B \text{ messages per day}$
- Peak QPS: $10B / 86400 \times 3 \text{ (peak factor)} \approx 350K \text{ writes/sec}$
- Storage: $10B \text{ msgs} \times 500 \text{ bytes} \approx 5 \text{ TB/day} \rightarrow \sim 1.8 \text{ PB/year}$
- Media: 10% msgs with $\sim 1MB \text{ media} \approx 1 \text{ PB/year}$ — served via CDN
- Bandwidth: $350K \times 500B \approx 175 \text{ MB/sec write throughput}$
- Takeaway: need horizontal scaling, sharding, async queues, and CDN for media

High Level Design (HLD)

How Does the Internet Work?

When you send a message in a WhatsApp-like chat system, data travels through several layers before reaching the recipient and back.

Understanding this path helps you design each component correctly.

- Client (App): mobile or web app initiates the request
- DNS (Domain Name System): converts chat.example.com into server IP addresses
- Load Balancer: distributes WebSocket connections across chat servers
- Chat Server: handles send/receive business logic
- Database / Cache: Cassandra for messages, Redis for sessions and hot data
- CDN (Content Delivery Network): serves profile pictures and media from the nearest edge server

Threads

A thread is the smallest unit of CPU execution within a process. Multiple threads share the process memory and run concurrently — essential for handling thousands of simultaneous chat connections on one server.

Without careful synchronization, concurrent threads can corrupt shared data through race conditions.

- Process vs Thread: a process is an independent program; threads within it share memory
- Multi-threading: many WebSocket connections handled concurrently in one chat server
- Thread pool: pre-created worker threads ready to process incoming messages
- Benefit: efficient I/O concurrency without spawning full new processes
- Race condition: two threads updating unread count simultaneously — one update can be lost
- Fix: use a lock (mutex) — only one thread updates shared data at a time; others wait

Pages & Virtual Memory

Physical RAM is real hardware memory shared by every running program. The OS uses pages and virtual memory so programs never overwrite each other's data — this is about isolation between programs (chat server vs database vs OS), not between chat users.

Each application gets its own private virtual address space. Programs only ever use virtual addresses; the OS translates every access via a page table (virtual page → physical page/frame). When RAM runs out, the OS swaps inactive pages to disk — a separate performance problem from isolation.

Virtual memory does NOT create extra RAM. For chat servers, each WebSocket connection consumes RAM for session state and buffers — capacity planning must account for this.

Core Concepts

Memory is managed in fixed-size blocks called pages (typically 4 KB). A virtual page is a slot in a program's private address book; a physical page (frame) is the real box in RAM where bytes are stored.

- Physical RAM: real memory chips; data lives in fixed 4 KB blocks called physical pages (frames)
- Problem — Isolation: without virtual memory, two programs could write the same physical address and corrupt each other
- Virtual memory: each program gets private virtual addresses (0, 1, 2...); it never knows physical RAM locations
- Page table: OS map per program — virtual page → physical page. Same virtual number in two apps maps to different physical pages

In Practice

Every read/write goes through translation. A page fault occurs when a virtual page is not in RAM — the OS loads it from disk, adding milliseconds of latency.

- Every read/write: program uses virtual page → CPU asks OS → page table lookup → access physical page in RAM
- Page fault: virtual page not in RAM → OS loads from disk → adds milliseconds of latency
- RAM full: OS moves inactive pages to disk (swap) — system keeps running but slower
- Chat capacity planning: $\text{RAM per connection} \times \text{max connections} \leq \text{server RAM budget}$
- Example: $50 \text{ KB/connection} \times 100\text{K connections} \approx 5 \text{ GB RAM}$ just for sessions

Horizontal vs Vertical Scaling

Scaling means growing capacity as demand grows. Vertical scaling (scale up) adds power to one machine — more CPU, RAM, or faster disks. Horizontal scaling (scale out) adds more machines behind a load balancer.

Stateless servers are the foundation of horizontal scaling. A stateless server stores no user-specific data in local memory — session state lives in Redis, messages in Cassandra. Any server can handle any request,

so the load balancer can freely distribute traffic. When one server fails, others take over without losing user context.

Modern chat systems scale horizontally — stateless API servers, shared session store in Redis, sharded databases — rather than relying on ever-larger single machines.

Vertical Scaling (Scale Up)

- Add more CPU/RAM to existing server — simple, no code changes
- Good for early stage or database primary node
- Hardware ceiling — cannot scale forever; single point of failure
- Example: upgrade chat DB from 32GB → 128GB RAM

Horizontal Scaling (Scale Out)

- Add more server instances behind load balancer — near-unlimited scale
- Fault tolerant — one server fails, others continue
- Requires stateless design — session in Redis, not local memory
- Example: 100+ stateless chat servers + sharded Cassandra

Serverful vs Serverless

Serverful (always-on servers) means you operate long-running processes that listen 24/7 for requests — EC2 instances, VMs, or Kubernetes pods. You own the infrastructure lifecycle: OS patching, runtime, deploys, monitoring, and scaling.

Serverless (functions on events) means you deploy small functions; the cloud provider runs them only when triggered by an event (HTTP request, queue message, file upload) and stops them when done. You manage code only — no servers to provision or patch.

Choose serverful for persistent, low-latency connections (WebSockets) and predictable workloads. Choose serverless for event-driven, short-lived, bursty tasks. Production chat systems often combine both — hybrid architecture.

Serverful — Always-On Servers

- Infrastructure: you provision EC2, VMs, or Kubernetes pods
- Execution: long-running process listens 24/7 for requests
- Operations: your team owns OS, runtime, deploys, patching, monitoring
- Scaling: add/remove instances via manual or auto-scaling groups
- Pricing: pay for server uptime — cost continues when idle
- Strengths: full control, WebSockets, predictable latency
- Limitations: ops overhead, idle capacity cost
- Chat use: WebSocket + REST API on K8s — always-on behind load balancer
- Example: EKS cluster with Go WebSocket chat-api — HPA scales pods 10 → 50

Serverless — Functions on Events

- Infrastructure: cloud provides ephemeral runtime — no servers to provision
- Execution: function runs on event (HTTP, queue, S3 upload), then stops
- Operations: you manage function code only — no OS or patching
- Scaling: cloud auto-scales concurrent executions on demand
- Pricing: pay per invocation + compute time — zero cost when idle
- Strengths: no idle spend, instant scale for bursty workloads
- Limitations: ~15 min max runtime, cold-start delay, no persistent WebSocket
- Chat use: thumbnail generation, push notification trigger, media transcoding
- Example: Lambda on S3 upload — 1,000 uploads spin up 1,000 parallel functions

SQL vs NoSQL Databases

SQL databases (RDBMS — Relational Database Management System) use fixed schemas and strong ACID guarantees — ideal for structured, relational data. NoSQL databases offer flexible schemas and horizontal scale — ideal for high-volume distributed workloads.

Chat systems typically use both: SQL for user profiles and contacts; NoSQL for billions of messages.

SQL (RDBMS)

- Fixed schema — tables, rows, columns with strict types
- Atomicity: a transaction completes entirely or rolls back — no half-updated transfers
- Consistency: database moves from one valid state to another — constraints always hold
- Isolation: concurrent transactions do not corrupt each other's partial work
- Durability: committed data survives crashes — written to disk before ACK
- Examples: PostgreSQL, MySQL
- Best for: user profiles, contacts, settings, billing
- Scaling: vertical + read replicas
- Chat: PostgreSQL for user metadata

NoSQL

- Flexible schema — document, key-value, or column stores
- Often eventual consistency (BASE: Basically Available, Soft state, Eventual consistency)
- Examples: Cassandra, MongoDB, Redis
- Best for: high-volume message history, presence, feeds
- Scaling: built for horizontal partition (sharding)
- Chat: Cassandra for message storage

In-Memory Databases

In-memory databases store data entirely in RAM (Random Access Memory) instead of on disk — sub-millisecond reads and writes. Because RAM is volatile, data can be lost on restart unless persistence is configured.

Redis and Memcached are the two most common production choices.

- Redis: rich data structures (lists, sets, sorted sets) plus optional disk persistence
- Publish/Subscribe (pub/sub): a messaging pattern where publishers emit events to channels without knowing subscribers. Subscribers listen to channels and receive events instantly — no polling. Chat uses pub/sub for typing indicators, online presence, and real-time fan-out to connected WebSocket servers
- AOF (Append-Only File): logs every write to disk — safer recovery, slightly slower
- RDB (Redis Database snapshot): periodic point-in-time backup — faster restart, small data-loss window
- Memcached: simple key-value, multi-threaded, no persistence — pure cache speed
- Use cases: session store, recent message cache, online status, rate-limit counters
- Chat: last 50 messages cached in Redis — opening a chat avoids slow DB round-trip

Data Replication & Migration

Replication copies data to multiple database nodes for fault tolerance and read scaling. Migration moves data from one database system to another — e.g. outgrowing MongoDB and moving to Cassandra — without hours of downtime.

- Leader-Follower (Primary-Replica): one leader accepts all writes; followers replicate asynchronously and serve reads. Simple and widely used — PostgreSQL read replicas, MySQL replication. Example: chat user profile writes go to primary; contact list reads hit replicas
- Leader-Leader (Multi-Master): multiple nodes accept writes — needs conflict resolution when two nodes update the same record simultaneously. Example: two users edit group name from different regions — system must merge or pick a winner (last-write-wins or CRDT)
- Why replicate: survive crashes, scale reads, place copies closer to users geographically

- Migration steps: dual-write to old and new DB → backfill historical data → switch reads → decommission old DB
- Chat replication: Cassandra stores 3 copies of each message across 3 data centers
- Chat migration: early startup on MongoDB; at scale, migrate to Cassandra for sharding and write throughput

Data Partitioning & Sharding

When one database server cannot hold all data or handle all writes, you split it. Partitioning divides data into logical subsets; sharding places those subsets on separate physical nodes.

The shard key (e.g. `chat_id`) decides which node owns each record — choose carefully.

- Hash partitioning: $\text{hash}(\text{chat_id}) \% N$ spreads chats evenly across N shards — best balance, avoids hot spots
- Range partitioning: chats A–M on shard 1, N–Z on shard 2 — simple but creates hot spots when popular data clusters in one range (e.g. all celebrity chats in shard 1)
- Shard key rule: all messages in one chat must land on the same shard — avoids cross-shard joins
- Hot shard problem: a celebrity group chat or viral channel concentrates all writes on one shard — that node becomes a bottleneck while others sit idle. Fixes: split the hot shard, use sub-sharding, or route hot keys to dedicated nodes
- Resharding: add nodes or split hot shards when storage exceeds ~80% or QPS spikes on one shard
- Chat: messages sharded by `chat_id` across Cassandra cluster

Data Consistency & Isolation Levels

In distributed systems, data exists on multiple nodes. Consistency (in plain language) means: when you read data, do you always see the latest write, or might you briefly see an older copy? Strong consistency = everyone sees the same answer immediately. Eventual consistency = copies may disagree briefly but converge over time.

Isolation levels (from SQL) answer: while one transaction is halfway done, can another transaction see its unfinished work? Higher isolation = safer but slower.

Chat messages tolerate slight delay; financial transactions do not.

- Strong consistency: every read returns the most recent write — all nodes agree (slower, safer)
- Eventual consistency: nodes may briefly disagree but converge — OK for online status and read receipts
- Read-your-writes: after you send a message, you always see it in your own chat — even if other users see it a moment later
- Read Committed: a transaction only sees data that has been fully committed — no half-finished writes visible
- Repeatable Read: once you read a row, repeated reads in the same transaction return the same value
- Serializable: strictest — transactions behave as if they ran one at a time
- Chat: eventual consistency for delivery status; serializable for payments

CAP Theorem

During a network partition (when nodes cannot talk to each other), a distributed system can guarantee at most two of three properties. Since network failures are inevitable in distributed systems, real designs choose CP or AP — not all three.

- Consistency (C): every read returns the latest write — all nodes agree on the same data
- Availability (A): every request receives a response — no 'try again later' errors
- Partition Tolerance (P): system continues operating despite network splits between nodes
- CA: only possible on a single node — not truly distributed

- CP example — banking: during partition, reject transfers rather than allow conflicting balances. Correctness beats uptime
- AP example — chat: during partition, accept messages locally in each region and sync when network heals. A silent, unavailable chat app is worse than slightly delayed sync
- PACELC extension: when no partition, still trade Latency vs Consistency

Consistency vs Availability

When two data centers lose connectivity, you must choose: reject writes to stay consistent, or accept writes locally and reconcile later.

For chat and messaging products, availability usually wins — a silent app that shows errors or refuses to send messages feels broken to users on flaky mobile networks. Slightly delayed read receipts or cross-region sync is acceptable; being unable to message is not.

Consistency-First (CP)

- Reject new writes until partition heals
- Read receipts only after all replicas confirm
- Strict global message ordering
- Risk: users see errors during outages — app may feel 'down'
- Used in: banking, inventory, ticket booking

Availability-First (AP)

- Accept writes locally; sync when network returns
- 'Delivered' shown quickly; 'read' may lag briefly
- Ordered within one chat; cross-region may differ briefly
- Benefit: messaging always works — no silent app on flaky networks
- Used in: chat, social feeds, notifications

Caching — Redis, Memcached & Policies

A cache is fast memory (usually RAM) sitting between your application and the database. Reading from a database might take 10–50 ms; reading from cache takes sub-milliseconds. On a cache hit you respond instantly; on a miss you read the DB and populate cache for next time.

You cannot cache everything. RAM is expensive and limited. Not all data is read frequently — caching cold data wastes memory. Cache invalidation is hard — stale data causes bugs. Some data must always be fresh (account balances). ~90% of chat reads are recent messages — perfect cache candidates.

Cache-Aside Pattern (Step-by-Step)

Cache-aside (lazy loading) is the most common pattern — the application manages both cache and DB.

1. **Read request arrives** — App checks cache first for the key (e.g. chat:123:messages)
2. **Cache hit** — Data found in cache → return immediately, skip DB
3. **Cache miss** — Data not in cache → query database
4. **Populate cache** — Write result to cache with TTL so next read is fast
5. **Return to client** — Send response — first request is slow, subsequent ones are fast
6. **Write path** — On new message: write to DB first, then invalidate or update cache entry

Redis vs Memcached

Both are production in-memory stores, but they serve slightly different roles.

Redis

- Rich data structures: strings, lists, sets, sorted sets, hashes
- Optional persistence: AOF and RDB snapshots
- Pub/sub built in — typing indicators, presence fan-out
- Single-threaded per instance — use Redis Cluster for scale
- Chat: sessions, recent messages, online status, rate limits

Memcached

- Simple key-value only — no lists or pub/sub
- No persistence — pure ephemeral cache
- Multi-threaded — good for plain GET/SET at high QPS
- Chat: profile cache, contact lists — data OK to rebuild from DB

Cache Write Policies

Write policies control whether cache and database update together or separately. High-volume messages favor speed; profile changes favor correctness.

- Write-through: write to cache AND DB together — always consistent, slower writes
- Write-back (write-behind): write cache first; flush to DB asynchronously — fastest, small data-loss risk on crash
- Write-around: write directly to DB; populate cache only on read — avoids caching data never read again
- Chat: write-back for high-volume messages; write-through for profile/settings changes

Cache Replacement (Eviction) Policies

When cache is full, an eviction policy decides which entries to remove. Good policies keep hot data and drop unlikely-to-be-read entries.

- LRU (Least Recently Used): evict longest-unaccessed entry — default for chat timelines
- LFU (Least Frequently Used): evict least popular — keeps frequently opened group chats
- Segmented LRU: separate hot/warm segments for active vs idle conversations
- TTL (Time To Live): auto-expire after fixed duration — online status (30s TTL)
- Chat: LRU for message history; TTL for presence indicators

Content Delivery Networks (CDN)

A CDN is a global network of edge servers storing copies of static and media files close to users. Instead of downloading from one central origin, users get content from the nearest edge node.

- Flow: upload to origin (S3) → CDN caches at edge nodes → user downloads from nearest edge
- Benefits: lower latency, reduced origin load
- DDoS protection: CDN absorbs attack traffic at the edge — malicious requests are filtered before reaching your origin servers. Cloudflare, Akamai, and AWS CloudFront all offer DDoS mitigation as a core feature
- Providers: CloudFront, Akamai, Cloudflare, Fastly
- On CDN: profile pics, shared photos/videos, stickers
- NOT on CDN: live chat text — delivered via WebSocket in real time
- Example: 2 MB photo in Mumbai served from Mumbai edge, not US origin

TCP & UDP — Transport Layer

At Layer 4 of the network stack, TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) move data between applications on different machines. IP (Internet Protocol) handles routing packets; TCP/UDP handle delivery semantics.

TCP is like registered mail — reliable, ordered, connection-oriented. Before sending data, TCP performs a 3-way handshake: (1) client sends SYN (synchronize), (2) server replies SYN-ACK

(synchronize-acknowledge), (3) client sends ACK (acknowledge). Only then does data flow. TCP guarantees delivery with retransmission, orders packets, and uses flow control.

UDP is like a postcard — fast, connectionless, no handshake. Packets may be lost, duplicated, or arrive out of order. Other transport protocols include SCTP (Stream Control Transmission Protocol) for telecom, and ICMP (Internet Control Message Protocol) for ping and network diagnostics.

Rule of thumb: TCP for messages, files, and APIs; UDP for live video, voice, and gaming where speed beats perfect delivery.

TCP — Reliable

- Connection-oriented: 3-way handshake (SYN → SYN-ACK → ACK) before data
- Guaranteed delivery with retransmission on packet loss
- Ordered packets; flow control prevents overwhelming receiver
- Higher latency due to handshake and ACK overhead
- Used for: HTTP, WebSocket, file transfer, email (SMTP)
- Chat: every text message delivered once, in order

UDP — Fast

- Connectionless: no handshake — send immediately
- No delivery or ordering guarantee — app handles loss if needed
- Minimal header overhead — lowest latency
- Used for: live video, gaming, DNS lookups, QUIC (HTTP/3 base)
- Chat: WebRTC voice/video tolerates occasional dropped frames

HTTP/1, HTTP/2, HTTP/3 & HTTPS

HTTP (Hypertext Transfer Protocol) is the foundation of web APIs — clients send requests (GET, POST, etc.) and servers return responses with status codes and bodies. HTTP/1.1 introduced persistent connections so one TCP connection could serve multiple requests, but each request still waited for the previous response (head-of-line blocking).

HTTP/2 multiplexes many requests and responses simultaneously over a single TCP connection, compresses headers, and enables server push. This dramatically improves page load and API throughput on high-latency networks.

HTTP/3 runs on QUIC (Quick UDP Internet Connections) — a protocol built on UDP instead of TCP. QUIC includes built-in TLS encryption, faster connection setup (0-RTT resumption), and eliminates TCP head-of-line blocking. Connection migration lets a mobile user switch Wi-Fi → 4G without reconnecting.

HTTPS is HTTP secured with TLS (Transport Layer Security). TLS encrypts all traffic between client and server — passwords, tokens, and message metadata cannot be read or tampered with in transit. Every production API should use HTTPS; browsers block mixed content and flag HTTP as insecure.

- HTTP/1.1: persistent connections — REST APIs for login, profiles, settings
- HTTP/2: multiplexing + HPACK header compression on one TCP connection
- HTTP/3: QUIC over UDP — faster setup, better on mobile and lossy networks
- QUIC: built-in encryption, connection migration (Wi-Fi → 4G without reconnect)
- TLS: encrypts data in transit — mandatory for auth tokens and sensitive APIs
- Chat: REST/HTTPS for profiles and history; real-time text uses WebSocket (TCP), not HTTP polling

WebSockets & Real-Time Alternatives

WebSocket upgrades an HTTP connection into a persistent full-duplex (two-way) channel over TCP. After an HTTP Upgrade handshake, both client and server can push data anytime — essential for chat delivery under 100ms.

Before WebSockets, engineers used workarounds. Short polling: client asks 'any messages?' every few seconds — simple but wasteful and slow. Long polling: server holds the request open until a message arrives, then client immediately opens a new one — better but still has connection overhead per message. Webhooks: server POSTs to a client URL on events — works for server-to-server, not mobile apps behind NAT.

WebSockets are the standard for browser and mobile real-time chat. Scaling requires handling thousands of open connections per server — sticky sessions or a connection registry in Redis.

- Upgrade flow: client sends HTTP Upgrade request → server responds 101 Switching Protocols → persistent bidirectional TCP channel
- Short polling: client repeatedly asks 'any messages?' — wastes bandwidth, 1–5s latency
- Long polling: server holds request until message arrives — lower latency than polling, still overhead per message
- Webhooks: server POSTs to registered URL on event — good for server-to-server integrations, not mobile clients
- Scaling: each user holds one open connection — use least-connections LB, sticky sessions, or Redis pub/sub for cross-server fan-out
- Chat: every online user maintains one WebSocket to a chat server cluster

WebRTC & Video Streaming

WebRTC (Web Real-Time Communication) enables peer-to-peer voice/video. STUN/TURN help traverse NAT/firewalls. Pre-recorded video uses HLS/DASH via CDN.

- WebRTC: P2P voice/video — signaling server helps peers connect
- STUN (Session Traversal Utilities for NAT): discovers public IP
- TURN (Traversal Using Relays around NAT): relays media when P2P fails
- VOD: HLS (HTTP Live Streaming) or DASH (Dynamic Adaptive Streaming over HTTP)
- Adaptive streaming: quality adjusts to network — 480p on 4G, 1080p on Wi-Fi
- Chat: calls use WebRTC; status videos use CDN streaming

Load Balancing

A load balancer (LB) distributes incoming requests across a pool of servers — essential for horizontal scaling. For chat, it must handle long-lived WebSocket connections, not just short HTTP requests.

- Round Robin: rotate requests across servers — good for stateless REST APIs
- Least Connections: route to server with fewest active connections — better for WebSockets
- Consistent Hashing: $\text{hash}(\text{user_id}) \rightarrow$ same server — minimal remap when scaling (see next topic)
- Health checks: stop sending traffic to failing servers automatically
- Layer 4 vs Layer 7: L4 routes by IP/port; L7 routes by URL, header, or cookie (SSL termination)
- Sticky sessions: same user → same chat server for open WebSocket connection

Consistent Hashing

Load balancers use consistent hashing to map users to servers — when you add or remove a server, only a small fraction of users need to reconnect. The same idea applies to caching (which Redis node holds a key) and sharding (which DB shard owns a chat_id).

Simple $\text{hash}(\text{key}) \% N$ remaps ~75% of keys when N changes — unacceptable at scale. Consistent hashing places keys and servers on a virtual ring; each key maps to the next server clockwise. Adding/removing a server only moves keys adjacent to that server on the ring.

- Relation to load balancing: $\text{hash}(\text{user_id}) \rightarrow$ stable server assignment across scale-up/down
- Relation to caching: Redis Cluster uses consistent hashing to distribute keys across nodes
- Relation to sharding: Cassandra token ring assigns partition keys to nodes

- Virtual nodes: each physical server gets multiple points on the ring — better load balance
- Chat: `hash(user_id)` → same chat server; minimal disruption when cluster grows from 10 → 20 servers

Proxy vs Reverse Proxy

A proxy is an intermediary that forwards traffic on behalf of someone. The direction matters — not request vs response.

Forward proxy sits on the client side: it acts on behalf of clients, hiding their identity from servers. Corporate networks use forward proxies for filtering and monitoring outbound traffic.

Reverse proxy sits on the server side: it acts on behalf of servers, sitting in front of backend services as the public entry point. Clients connect to the reverse proxy; it routes internally to chat servers, handles SSL/TLS termination, caching, and rate limiting.

Chat backends use reverse proxy (e.g. Nginx) — users see one address; the proxy distributes to the cluster behind it.

Forward Proxy (Client Side)

- Sits in front of clients — between client and internet
- Hides client IP from destination servers
- Used: corporate web filtering, geo-restricted access, anonymity
- Not typical in chat backend architecture

Reverse Proxy (Server Side)

- Sits in front of servers — public traffic hits proxy first
- SSL/TLS termination, load balancing, caching, rate limiting
- Example: Nginx as single entry point to chat server cluster
- Clients see one server; proxy routes internally to backend pool

Rate Limiting

Rate limiting caps requests per user/IP/time window — prevents spam and protects servers from overload.

- Token Bucket: tokens refill at fixed rate; allows short bursts
- Sliding Window: count requests in rolling time window
- HTTP 429 Too Many Requests when exceeded
- Distributed: Redis counter shared across all API servers
- Chat: max 100 messages/minute per user

Message Queues & Async Processing

Queues sit between producer and consumer — sender gets instant ACK while delivery, notifications, and analytics happen asynchronously in background.

- Kafka: distributed log, high throughput, ordering within partition
- RabbitMQ: traditional broker with routing and dead-letter queues
- Queue vs Pub/Sub: queue delivers each message to one consumer; pub/sub broadcasts a copy to every subscriber
- Benefits: spike absorption, retry on failure, independent deployment
- Chat: Kafka for message events; RabbitMQ for push notifications

Event-Driven Architecture & Saga Pattern

Event-driven architecture (EDA) decouples services through events instead of direct synchronous calls. When a user sends a message, the chat service publishes a `MessageSent` event — notification, analytics, and search services react independently without the chat service knowing about them.

Choreography: each service listens and reacts on its own — no central controller. Simple for small flows but hard to trace. Orchestration: a central coordinator directs each step in order — easier to monitor but adds a dependency.

The Saga pattern coordinates multi-step distributed transactions without a single ACID database. Each step commits locally; if a later step fails, compensating transactions undo previous steps. Example: media upload saga — (1) store raw file in S3, (2) transcode video, (3) update message record, (4) notify recipient. If transcode fails, delete the S3 file and mark message as failed.

- MessageSent event → notification service sends push, analytics increments counters, search indexes content
- Choreography: services subscribe to events independently — loose coupling, harder debugging
- Orchestration: Saga coordinator calls each service in sequence — clear flow, single point to monitor
- Compensating transaction: on failure, run reverse operations — delete uploaded file, refund payment
- Chat: group message triggers pub/sub fan-out event; each member's notification service reacts

REST vs gRPC vs GraphQL

REST (Representational State Transfer) is an architectural style using HTTP methods (GET, POST, PUT, DELETE) with JSON payloads. Universal, easy to debug, works in browsers — the default for public APIs.

gRPC (Google Remote Procedure Call) is a high-performance RPC framework using HTTP/2 and Protocol Buffers (binary serialization). Services call remote functions as if local — 5–10× faster than JSON REST for internal microservice communication.

GraphQL is a query language where the client specifies exactly which fields it needs in one request. Mobile apps fetch home screen data (profile + recent chats + unread counts) in a single round trip instead of multiple REST calls.

- REST: HTTP + JSON — public APIs, third-party integrations, browser-friendly
- gRPC: HTTP/2 + Protobuf — internal service-to-service calls, streaming, strong typing
- GraphQL: client-driven queries — mobile apps with varied data needs, reduces over-fetching
- Chat: REST for user profiles and settings; gRPC between chat ↔ notification services; GraphQL for mobile home screen

Idempotency

An operation is idempotent if executing it once or many times produces the same result. POST /messages with the same idempotency key sent twice due to network retry must not create two messages — the server returns the original response on the duplicate.

Without idempotency, flaky mobile networks cause duplicate messages, double charges, and duplicate account creation.

- Client sends X-Idempotency-Key (UUID) on every write request
- Server stores processed keys in Redis; duplicate key → return original response, skip re-processing
- TTL: keys expire after ~24 hours to limit storage
- Critical for: message send, payments, account creation, any non-safe HTTP method
- Chat: every POST /messages includes idempotency key — retry-safe delivery

Monolith vs Microservices

Monolith: one codebase, one deployment — all features (auth, messaging, media, notifications) in a single application. Simple to build and debug early on.

Microservices: independent services with own databases, deploy cycles, and scaling. Scale only the bottleneck — add chat servers without scaling auth.

Start with a monolith; extract services when a specific part becomes a bottleneck or needs independent release cadence.

Monolith

- Single codebase and deployment — simple early on
- Scale entire app together — even unused features
- One bug can crash everything
- Early startup: one app server for auth, messaging, media, notifications

Microservices

- Independent services, own databases, independent deploys
- Scale only bottleneck — add chat servers without scaling auth
- Failure isolated per service — notification outage doesn't kill messaging
- Mature chat: auth, messaging, media, notifications, search as separate services

Migration to Microservices

Imagine a chat startup that launches as a monolith — one Node.js app handles login, messaging, push notifications, and media uploads. It works fine for 10K users.

At 1M users, message delivery slows because notification fan-out blocks the main thread. Media uploads compete for the same server memory. The team identifies messaging and notifications as separate bottlenecks.

They extract the notification service first (clear boundary, low risk) using the Strangler Fig pattern: run old and new side by side, dual-write events, shift traffic gradually, decommission monolith code for that feature. Repeat for media, search, and auth.

1. **Identify bounded contexts** — Auth, messaging, media, notifications — each with distinct scaling needs
2. **Extract one service** — Start with notifications — publish MessageSent events instead of inline calls
3. **Dual-run** — Old monolith and new service both process events; compare outputs for correctness
4. **Shift traffic** — Route 10% → 50% → 100% of notification work to new service
5. **Decommission** — Remove notification code from monolith once new service is stable
6. **Repeat** — Extract media service next, then search, then auth — one at a time

Single Point of Failure (SPOF)

A SPOF is any single component whose failure takes down the entire system or a critical path. Good architecture eliminates SPOFs at every layer with redundancy.

- SPOF example: single database with no replicas — DB crash loses all chat data. Fix: leader-follower replication with automatic failover
- SPOF example: one load balancer — LB crash drops all traffic. Fix: active-passive LB pair with virtual IP failover
- SPOF example: single Redis instance for sessions — restart logs everyone out. Fix: Redis Sentinel or Redis Cluster with replicas
- SPOF example: single DNS provider — DNS outage makes chat.example.com unreachable. Fix: multiple DNS providers, low TTL, anycast
- SPOF example: one availability zone — datacenter fire kills everything. Fix: multi-region deployment with Cassandra RF=3 across zones
- Principle: no single hardware failure, deploy, or config mistake should kill messaging for all users

Cascading Failures, Circuit Breaker & Bulkhead

A cascading failure starts when one slow or failing component causes dependent services to exhaust resources and fail in a chain. Example: notification DB slows → chat service threads block waiting → thread pool full → chat API stops responding → total outage from a non-critical dependency.

Resilience patterns break the chain before it spreads. Timeouts prevent indefinite waiting. Circuit breakers fail fast when a downstream service is unhealthy. Bulkheads isolate resource pools so one failing area cannot consume all threads or connections.

- Circuit breaker states: Closed (normal) → Open (failing — reject calls immediately) → Half-Open (test one request to check recovery)
- Bulkhead: separate thread pools for messaging, media, notifications — notification slowness cannot starve message delivery
- Timeout: never wait indefinitely on downstream calls — set 200ms for notification, 2s for media
- Retry with exponential backoff: 1s, 2s, 4s on transient Kafka failure — avoid thundering herd
- Chat: circuit breaker on notification service — if push is down, messages still deliver via WebSocket

Docker & Containerization

Docker packages an application and all its dependencies (runtime, libraries, config) into a portable image. The same image runs identically on a developer laptop, staging, and production — eliminating 'works on my machine' problems.

A container is a lightweight isolated process sharing the host OS kernel — far faster to start than a full VM. Kubernetes (K8s) orchestrates containers at scale: auto-scaling, self-healing (restart crashed pods), rolling deploys with zero downtime, and service discovery.

- Container vs VM: container shares OS kernel — starts in seconds; VM includes full OS — starts in minutes
- Docker image: immutable artifact built in CI — tagged and pushed to registry (ECR, GCR, Docker Hub)
- K8s Deployment: declares desired state (10 chat-api pods) — controller maintains it automatically
- HPA (Horizontal Pod Autoscaler): scales chat pods 10 → 100 during peak based on CPU or custom metrics
- CI/CD: git push → build Docker image → push to registry → K8s rolling update → health check → traffic shift
- Chat: each microservice (auth, messaging, notify) runs as separate containerized deployment

Object Storage (S3)

Object storage treats files as objects in a flat namespace. AWS S3 (Simple Storage Service), GCS, Azure Blob offer massive scale and durability for media.

- Use cases: profile pics, shared photos, voice notes, video messages
- Flow: upload to S3 → CDN caches → recipient downloads from edge
- Pre-signed URL: temporary authenticated direct upload/download link
- Durability: S3 offers 99.999999999% (11 nines)
- Chat: media in S3; server stores metadata + URL only

Logging

Logging captures timestamped records of what happened — service, user, action, errors. Centralized logs let you trace 'message not delivered' across hundreds of servers.

- Structured JSON: timestamp, level, service, trace_id, user_id
- ELK Stack: Elasticsearch, Logstash, Kibana
- Log levels: DEBUG → INFO → WARN → ERROR → FATAL

- Best practice: log metadata, not message body if E2E encrypted

Metrics, Monitoring & Distributed Tracing

Observability triad: metrics (numbers over time), logs (events), traces (request path across services). Together they explain system health and failures.

- Prometheus: pull-based metrics collection and alerting
- Grafana: dashboards for throughput, p99 latency, error rate
- Distributed tracing: trace_id propagated Client → Chat → Kafka → Notify
- Tools: Jaeger, Zipkin, AWS X-Ray
- p99 latency: 99% of requests complete within threshold
- Alert when message delivery p99 exceeds 500ms

Anomaly Detection & Disaster Recovery

Anomaly detection flags unusual patterns — spike in failed deliveries may mean outage or attack. DR plans use RTO (Recovery Time Objective) and RPO (Recovery Point Objective).

- RTO: max acceptable downtime — chat target ~5 minutes
- RPO: max acceptable data loss — chat target zero (no message loss)
- Strategy: multi-region Cassandra + S3 cross-region backup
- Failover drills: simulate datacenter failure regularly
- Chaos engineering: inject failures to test resilience before real outages

Security — Auth, Encryption & Access Control

Authentication verifies who you are — login with password, OTP, or OAuth ('Login with Google'). Authorization determines what you are allowed to do — only group members can read group messages.

Encryption protects data from being read by unauthorized parties. TLS encrypts data in transit (HTTPS, WSS); AES-256 encrypts data at rest on databases and object storage.

Chat adds E2E (end-to-end) encryption so the server cannot read message bodies — only sender and recipient hold decryption keys.

- JWT (JSON Web Token): signed token sent with every API/WebSocket request — server verifies signature without DB lookup
- OAuth 2.0: delegate authentication to Google/Apple — your app gets a token without storing passwords
- ACL (Access Control List): per-resource permissions — user X can read chat Y, admin can ban user Z
- TLS: encrypts data in transit — mandatory for all production APIs
- Encryption at rest: AES-256 on DB columns and S3 buckets
- E2E: Signal Protocol — keys exchanged client-to-client; server stores ciphertext only

Push vs Pull

Push means the server sends data to the client as soon as it is available — no waiting. Pull means the client repeatedly asks the server for updates.

Push is lower latency and more efficient for real-time features. Pull is simpler to implement and works through firewalls more easily, but wastes bandwidth and adds delay.

Push (Server → Client)

- Server initiates delivery when data is ready
- Low latency — message arrives in milliseconds
- Efficient — no wasted empty requests
- Requires persistent connection (WebSocket) or platform push (APNs, FCM)

- Chat: WebSocket push for live messages; FCM/APNs for offline push notifications

Pull (Client → Server)

- Client polls server on a timer — 'any new messages?'
- Higher latency — depends on poll interval (1–30 seconds)
- Wasteful — most polls return empty when no new data
- Simple — works with plain HTTP, no special infrastructure
- Chat: fallback pull for clients that cannot maintain WebSocket

System Design Trade-offs

No perfect architecture — only trade-offs aligned with requirements. Name the trade-off and justify your choice.

Trade-off	Option A	Option B	Chat Choice
Push vs Pull	Server pushes instantly	Client polls	Push via WebSocket
Consistency vs Availability	Strong (CP)	High (AP)	AP — always on
SQL vs NoSQL	Structured, ACID	Flexible, scalable	SQL users; NoSQL messages
Memory vs Latency	More cache = faster	Less cache = cheaper	Redis for hot data
Monolith vs Microservices	Simple to build	Independent scale	Start monolith

Popular HLD Systems to Practice

Practice designing these systems on paper — each teaches different concepts.

- YouTube: CDN, video transcoding, blob storage, recommendations
- Twitter/X: fan-out on write/read, timeline generation, cache
- WhatsApp: WebSockets, message queues, E2E encryption, presence
- Uber: geolocation, real-time matching, surge pricing, maps
- Amazon: inventory, payments, consistency, recommendations
- Google Drive: file sync, conflict resolution, chunk storage
- Netflix: CDN streaming, microservices, recommendations, A/B testing
- Instagram: feed generation, media storage, CDN, social graph
- Zoom: WebRTC, SFU/MCU, low-latency video, screen sharing
- Airbnb: search, booking, payments, geolocation, reviews

Search Autocomplete — HLD Glimpse

Autocomplete suggests matches as user types. At scale requires trie (prefix tree) for fast lookup and Redis cache for popular prefixes.

- Trie: each path from root spells a prefix — $O(m)$ lookup for m characters typed
- Cache top 1000 popular prefixes in Redis
- Rank by frequency, recency, personal relevance (contacts first)
- Rebuild index nightly; target latency under 50ms
- Chat: contact search uses trie + cache

Low Level Design (LLD)

OOP — Object-Oriented Programming

OOP organizes code into objects that bundle data (fields) and behavior (methods). Four pillars model real-world entities in LLD problems.

- Encapsulation: hide internal state behind methods — Chat class hides how messages are stored; callers use `sendMessage()`
- Abstraction: expose only what callers need — User sees `sendMessage(text)`, not DB connection details
- Inheritance: reuse common behavior — GroupChat extends Chat, adding `addMember()` and `removeMember()`
- Polymorphism: same interface, different behavior — `Notification.send()` works for `PushNotification`, `EmailNotification`, `SMSNotification`
- LLD interview tip: nouns → classes, verbs → methods, relationships → inheritance or composition

SOLID Principles

Five principles for maintainable, extensible code — essential in LLD interviews and production codebases.

- S — Single Responsibility: `MessageValidator` only validates; `MessageStore` only persists — one reason to change each
- O — Open/Closed: add `EncryptedMessage` type without modifying `MessageSender` — extend via new class, not edits
- L — Liskov Substitution: `DirectChat` and `GroupChat` both work wherever Chat is expected — no surprise behavior
- I — Interface Segregation: `Readable` and `Writable` as separate interfaces — read-only clients don't depend on write methods
- D — Dependency Inversion: `MessageService` depends on `MessageRepository` interface, not PostgreSQL directly — swap DB in tests

Coupling, Cohesion & Composition

Coupling measures how much one module depends on another. Low coupling means modules can change independently — `MessageService` doesn't know if storage is Redis or PostgreSQL.

Cohesion measures how focused a module is. High cohesion means each class does one thing well — `UnreadCounter` only tracks unread counts, not authentication.

Composition ('has-a') builds objects from parts; inheritance ('is-a') shares behavior through a parent class. Prefer composition unless a genuine subtype relationship exists — GroupChat HAS-A list of Members; it IS-A Chat only if it truly substitutes everywhere Chat is used.

- Loose coupling: `MessageService` depends on `NotificationInterface`, not `FirebasePushService` directly
- High cohesion: `MediaUploader` handles upload only — not validation, not notification
- Composition: `ChatRoom` HAS-A `List<Message>` and `List<User>` — flexible, no fragile inheritance chains
- Inheritance: `GroupChat` IS-A Chat — only when `GroupChat` can replace Chat everywhere without breaking
- Anti-pattern: `MessageService` that also handles auth, media, and notifications — low cohesion, high coupling

Design Patterns

A design pattern is a proven, reusable solution to a recurring design problem in code. Patterns are not libraries — they are templates you adapt to your context. Gang of Four (GoF) categorized them into creational (object creation), structural (composition), and behavioral (communication).

Creational Patterns

Control how objects are instantiated — hide construction complexity and enforce constraints.

- Singleton: exactly one instance — `DatabaseConnectionPool`, `Logger` (use sparingly — hard to test)

- Factory Method: `MessageFactory.create('text') → TextMessage`, `create('media') → MediaMessage`
- Abstract Factory: `NotificationFactory` for iOS/Android push implementations
- Builder: `MessageBuilder.setText('hi').setReplyTo(id).setMedia(url).build()` — readable construction
- Prototype: clone a template `Message` for broadcast to 1000 group members

Structural Patterns

Compose classes and objects into larger, flexible structures.

- Adapter: wrap `LegacySmsGateway` to match `NotificationService` interface
- Facade: `ChatFacade.sendMessage()` hides DB + cache + queue complexity behind one method
- Decorator: `EncryptedMessageDecorator` wraps `Message` and adds encrypt/decrypt
- Proxy: `LazyMessageProxy` loads message body from S3 only when user scrolls to it
- Composite: `ChatItem` interface implemented by `Message` and `MessageGroup` — uniform treatment

Behavioral Patterns

Define how objects communicate, handle events, and manage workflow.

- Observer: `MessageSubject` notifies all `ChatObserver` subscribers on new message
- Strategy: `PushStrategy`, `EmailStrategy`, `SMSStrategy` — interchangeable at runtime
- State: Message transitions `Sent → Delivered → Read` — each state defines allowed actions
- Chain of Responsibility: `SpamFilter → ProfanityFilter → LengthFilter → Deliver`
- Command: `SendMessageCommand` encapsulates send action with undo/delete support
- Mediator: `ChatRoomMediator` coordinates Users — they don't reference each other directly

Concurrency & Thread Safety

Multiple threads accessing shared mutable state need explicit protection. Race conditions, deadlocks, and lost updates are among the most common and hardest-to-debug production bugs.

In a chat server handling 10K concurrent `WebSocket` connections, threads share connection registries, unread counters, and message buffers — all require careful synchronization.

- Race condition: two threads increment `unreadCount` for same chat — both read 5, both write 6, one update lost
- Fix — Mutex/Lock: `synchronized(chatId) { unreadCount++ }` — only one thread updates at a time
- Read-Write lock: many threads read online status simultaneously; only one writes on connect/disconnect
- `ConcurrentHashMap`: thread-safe online user registry without external locking
- Producer-Consumer: `WebSocket` threads enqueue incoming messages; worker pool dequeues and persists
- Deadlock: Thread A locks `chat1` then `chat2`; Thread B locks `chat2` then `chat1` — both wait forever. Fix: always lock in consistent order
- `AtomicInteger` / atomic operations: lock-free counters for high-contention metrics

UML Diagrams

UML (Unified Modeling Language) communicates design before coding — standard in LLD interviews.

- Class diagram: classes, attributes, relationships
- Sequence diagram: message flow `User → ChatService → Repository → DB`
- State diagram: Message `Composing → Sent → Delivered → Read`
- Activity diagram: `receive → validate → store → notify → ack`
- Use case diagram: User sends msg; Admin bans user

API Design — Request/Response Modeling

Clean API design makes systems easier to use, test, and evolve. Consistent naming, validated DTOs (Data Transfer Objects), standard error formats, and cursor-based pagination are hallmarks of production APIs.

Version APIs from day one (/v1/) so you can introduce breaking changes in /v2/ without breaking existing clients.

- RESTful URLs: POST /v1/chats/{chatId}/messages — nouns, not verbs; plural collections
- Request DTO: SendMessageRequest { text, mediaId, replyToId, idempotencyKey }
- Response DTO: SendMessageResponse { messageId, status, timestamp, serverSeq }
- Standard errors: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Rate Limited — with JSON body { error, message, code }
- Cursor pagination: GET /messages?cursor=abc123&limit=50 — stable for live feeds (offset breaks when new messages arrive)
- Idempotency: X-Idempotency-Key header on all write endpoints
- Versioning: /v1/ prefix; add optional fields for backward compatibility; never remove fields in same version

Clean Code & Avoiding God Classes

A God Class is a single class that knows too much and does too much — it handles authentication, messaging, media, notifications, and billing in one 5000-line file. God Classes violate Single Responsibility, are impossible to test in isolation, and become merge-conflict magnets.

Clean code is readable, testable, and focused. Split God Classes into single-purpose services; extract shared logic once (DRY); keep solutions simple (KISS); don't build features you don't need yet (YAGNI).

- DRY (Don't Repeat Yourself): extract shared validation into MessageValidator — one place to fix bugs
- KISS (Keep It Simple, Stupid): start with direct DB call; add cache only when profiling shows need
- YAGNI (You Aren't Gonna Need It): don't build video calling until messaging works and is scaled
- Single-purpose classes: AuthService, MessageService, MediaService, NotificationService — each testable alone
- God Class fix: identify responsibilities → extract each into its own class → inject dependencies

MVC, MVP, MVVM & Repository Pattern

Separating UI from business logic is critical for testability and maintainability. MVC (Model-View-Controller) is the most widely used pattern — it keeps presentation, data, and control flow distinct.

The Repository pattern hides storage details behind an interface — swap PostgreSQL for Cassandra in tests without changing business logic.

MVC (Model-View-Controller)

Model holds data and business rules. View renders UI. Controller handles user input, updates Model, and selects View.

- Model: Message, Chat, User entities with validation logic
- View: renders chat list and message bubbles (HTML template or mobile UI)
- Controller: receives 'send message' action → validates → calls MessageService → updates View
- Best for: server-rendered web apps, early mobile apps

MVP (Model-View-Presenter)

View is passive (dumb) — it only renders what Presenter tells it. Presenter handles all logic and talks to Model.

- View: displays messages; forwards taps to Presenter
- Presenter: fetches messages from Model, formats for View, handles errors
- Better testability: mock View, test Presenter logic in isolation
- Best for: Android apps (before MVVM adoption)

MVVM (Model-View-ViewModel)

View binds to ViewModel properties — UI auto-updates when data changes (data binding). ViewModel holds presentation state, no View reference.

- ViewModel: exposes observable properties — unreadCount, messageList, isOnline
- View: binds to ViewModel — no logic, just layout and bindings
- Best for: modern mobile (SwiftUI, Jetpack Compose) and SPA frameworks (React, Vue)

Repository Pattern

Abstracts data access behind an interface — business logic depends on MessageRepository, not SQL.

- Interface: MessageRepository { save(), findByChatId(), delete() }
- Implementations: PostgresMessageRepository, CassandraMessageRepository, InMemoryMessageRepository (tests)
- Business code unchanged when storage changes — swap implementation via dependency injection

Schema Design, Testability & Domain Modeling

Schema design defines tables, relationships, indexes, and constraints. Good schema balances normalization (avoid duplicate data) with query performance (strategic denormalization).

Domain models represent business concepts in code — MessageStatus enum, Chat entity. DTOs carry data across API boundaries. Dependency injection lets you unit test business logic with mock repositories — no real database needed.

- Core tables: users, chats, chat_members, messages, message_status
- Junction table: chat_members(user_id, chat_id, role) — many-to-many without duplicating user data
- Index: messages(chat_id, created_at DESC) — fast 'load last 50 messages' query
- Composite index: message_status(message_id, user_id) — fast per-user delivery status lookup
- Enums: MessageStatus { SENT, DELIVERED, READ, FAILED } — type-safe, no magic strings
- DTO vs domain: SendMessageRequest (API) vs Message entity (business logic) — separate concerns
- Testability: inject MockMessageRepository — test MessageService.send() without PostgreSQL
- Soft delete: messages.deleted_at timestamp — recoverable delete, audit trail

Common LLD Problems to Practice

Code these in your language to build OOP and pattern skills.

- Tic-Tac-Toe / Chess: board, rules, turn management, win detection
- Splitwise: expense splitting, balances, simplify debts
- Parking Lot: multi-level, vehicle types, slot allocation
- Elevator System: multiple lifts, scheduling, direction optimization
- Notification System: push/email/SMS channels, priority, retry
- Food Delivery: restaurant, order, agent, tracking, payment
- Movie Ticket Booking: seat selection, concurrency
- URL Shortener: encoding, cache, analytics (also HLD classic)
- Logging Framework: levels, appenders, formatters, singleton
- Rate Limiter: token bucket, sliding window (also HLD classic)

HLD vs LLD — Same Problem, Two Views

HLD asks how components connect at scale. LLD asks how classes are structured. Same product needs both views.

Aspect	HLD View (Chat)	LLD View (Parking Lot)
Core Question	How do components connect at scale?	How are classes structured?
Components	Chat Svc, Kafka, Redis, Cassandra	ParkingLot, Slot, Vehicle, Ticket
Data Flow	Client → LB → Chat → Queue → DB	Vehicle → findSlot() → park()
Primary Concern	Availability, latency, throughput	Extensibility, testability
Diagram	Service architecture boxes	Class diagram with inheritance

Study Guide & Summary

Study Roadmap & Resources

Combine theory with weekly practice — one HLD design and one LLD coding problem per week. Mock interviews provide the fastest feedback.

- HLD: Grokking SDI, System Design Primer (GitHub), ByteByteGo
- Practice HLD: YouTube, Uber, WhatsApp, Twitter — 1/week on paper
- LLD: Head First Design Patterns, Refactoring Guru, LeetCode LLD
- Practice LLD: Parking Lot, Elevator, Splitwise in your language
- Foundations: TCP/UDP, DNS, indexing, CAP, consistent hashing
- Mock interviews: Pramp, Interviewing.io, peers — 45 min weekly
- Always start with requirements and estimations before architecture

Summary & Key Takeaways

System Design spans HLD (architecture at scale) and LLD (code structure). Use the 6-step approach, name trade-offs explicitly, and practice with real systems.

- HLD: scalability, reliability, trade-offs, distributed building blocks
- LLD: OOP, SOLID, patterns, concurrency, clean code, API/schema design
- Approach: Requirements → Estimation → API → Architecture → Deep Dive → Trade-offs
- Every decision is a trade-off — consistency vs availability, memory vs latency
- Start simple (monolith) — add complexity only when requirements demand it
- Running example: WhatsApp-like chat system ties HLD concepts together throughout